6	A music player system uses a Circular Queue to manage the playlist. Songs are played in the order they were added. Once the end of the playlist is reached, the player loops back to the beginning and continues playback in a circular manner. Implement the following operations using a Circular Queue: a. Insert a Song into the Circular Queue b. Add a new song to the playlist. Ensure circular behavior is maintained when the queue is full. c. Delete (Play and Remove) a Song from the Circular Queue d. Remove the currently playing song from the queue (simulate playback completion). e. Demonstrate Overflow and Underflow in the Circular Queue Show appropriate error messages when: Trying to add a song to a full queue (Overflow) Trying to play/delete a song from an empty queue (Underflow) Display the Current Playlist (Status of Circular Queue) Show all the songs currently in the queue in order, maintaining the circular structure.

```
#include <stdio.h>
#include <string.h>
#define MAX 5
char playlist[MAX][50];
int front = -1, rear = -1;

int isFull() {
   return (front == (rear + 1) % MAX);
}

int isEmpty() {
   return (front == -1);
}

void addSong(char song[]) {
   if (isFull()) {
      printf("\nOverflow! Playlist is full.\n");
      return;
   }
   if (isEmpty())
      front = rear = 0;
   else
      rear = (rear + 1) % MAX;
   strcpy(playlist[rear], song);
   printf("Added: %s\n", song);
}

void playAndRemove() {
   if (isEmpty()) {
      printf("\nUnderflow! No song to play.\n");
      return;
   }
   printf("Playing & Removing: %s\n", playlist[front]);
   if (front == rear)
      front = rear = -1;
   else
      front = (front + 1) % MAX;
}

void showPlaylist() {
   if (isEmpty()) {
```

```
        printf("\nPlaylist is empty.\n");
        return;
    }
    printf("\nCurrent Playlist:\n");
    int i = front;
    while (1) {
        printf("%s\n", playlist[i]);
        if (i == rear) break;
        i = (i + 1) % MAX;
    }
    printf("\n");
}

int main() {
    int choice;
    char song[50];
    while (1) {
        printf("\n=== MUSIC PLAYLIST MENU ===\n");
        printf("1. Add Song\n");
        printf("2. Play & Remove Song\n");
        printf("3. Display Playlist\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Enter song name: ");
                scanf(" %[^\n]", song);   // read full string
                addSong(song);
                break;
            case 2:
                playAndRemove();
                break;
            case 3:
                showPlaylist();
                break;
            case 4:
                printf("Exiting...\n");
                return 0;
            default:
                printf("Invalid Choice!\n");
        }
    }
    return 0;
}
```